Pulsejet Combustion Instability Model

MATLAB source listing: pulsejet_instability_model.m
255 lines | Hopf oscillator + 2D acoustic wave equation + diffusion field

Chamber: 0.5 m x 0.1 m rectangle, closed left end, open right end
Total simulated time: 10 ms (about 808 time steps)

Requires MATLAB R2018b or newer (uses xline, caxis, parula).
See README and THEORY PDFs for usage, assumptions and the underlying equations.

pulsejet_instability_model.m

```
%% ========================================================================
% SIMPLIFIED Pulsejet Combustion Instability Model
% Clean, working version
% ========================================================================

clear; close all; clc;

%% ==================== 1. SETUP ==========================================
% Geometry (based on paper)
Lx = 0.5;      % Chamber length (m)
Ly = 0.1;      % Chamber width (m)

% Wave properties
c = 340;       % Speed of sound (m/s)

% Instability parameters
mu = 0.05;     % Control parameter (positive for instability)
omega0 = 1000; % Frequency (rad/s)
beta = 0.1;    % Nonlinear saturation

% Numerical parameters
Nx = 50;       % Points in x
Ny = 20;       % Points in y
dx = Lx/(Nx-1);
dy = Ly/(Ny-1);
dt = 0.8*min(dx,dy)/c;  % CFL condition
T_total = 0.01;         % Total time (s)
Nt = round(T_total/dt);

% Grid
x = linspace(0, Lx, Nx);
y = linspace(0, Ly, Ny);
[X, Y] = meshgrid(x, y);

%% ==================== 2. INITIAL CONDITIONS ============================
% Pressure field - Gaussian pulse at ignition
x0 = Lx/3; y0 = Ly/2;
sigma = 0.05;
P0 = 1e5;  % 100 kPa initial overpressure

P = zeros(Ny, Nx);
P_prev = P;

for i = 1:Nx
    for j = 1:Ny
        r = sqrt((x(i)-x0)^2 + (y(j)-y0)^2);
        P(j,i) = P0 * exp(-r^2/(2*sigma^2));
    end
end

% Instability amplitude field (KS variable)
phi = zeros(Ny, Nx);

%% ==================== 3. HOPF OSCILLATOR ================================
% dz/dt = (mu + iω)z - β|z|^2 z
z = 0.001 + 0.001i;  % Small initial perturbation
z_history = zeros(1, Nt);
t_history = zeros(1, Nt);

%% ==================== 4. ANIMATION SETUP ================================
figure('Position', [100 100 1200 500]);

% Subplot 1: Pressure
subplot(1,3,1);
h_p = surf(X, Y, P, 'EdgeColor', 'none');
view(2);  % Top view
axis tight;
colorbar;
caxis([-5000 5000] + 101325);
title('Pressure (Pa)');
xlabel('Length (m)');
ylabel('Width (m)');
colormap(jet);

% Subplot 2: Instability
subplot(1,3,2);
h_phi = surf(X, Y, phi, 'EdgeColor', 'none');
view(2);
```

```
axis tight;
colorbar;
title('Instability Amplitude');
xlabel('Length (m)');
ylabel('Width (m)');
colormap(parula);

% Subplot 3: Time traces
subplot(1,3,3);
hold on;
h_zplot = plot(0, real(z), 'b-', 'LineWidth', 2);
h_pplot = plot(0, P(round(Ny/2), round(Nx/2)), 'r-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Amplitude');
legend({'Hopf Oscillator', 'Center Pressure'}, 'Location', 'best');
grid on;
title('Time Evolution');

%% ==================== 5. MAIN SIMULATION LOOP ========================
fprintf('Running simulation...\n');
fprintf('Time steps: %d, dt = %.2e s\n', Nt, dt);

% Store for plotting
pressure_center = zeros(1, Nt);
time_points = zeros(1, Nt);

for n = 1:Nt
    t = n*dt;
    time_points(n) = t;

    % ========== Hopf oscillator update ==========
    % Euler integration
    z = z + dt * ((mu + 1i*omega0)*z - beta*z*abs(z)^2);
    z_history(n) = z;

    % Instability amplitude affects combustion
    A_inst = real(z);

    % ========== KS equation (simplified) ==========
    % Simple diffusion with source
    phi_new = phi;
    for i = 2:Nx-1
        for j = 2:Ny-1
            % Laplacian
            lap_phi = (phi(j,i+1) + phi(j,i-1) - 2*phi(j,i))/dx^2 + ...
                      (phi(j+1,i) + phi(j-1,i) - 2*phi(j,i))/dy^2;

            % Simple KS-like update
            phi_new(j,i) = phi(j,i) + dt * (0.01*lap_phi + ...
                           A_inst * sin(omega0*t) * exp(-((x(i)-x0)^2 + (y(j)-y0)^2)/0.01));
        end
    end
    phi = phi_new;

    % ========== 2D Wave equation ==========
    P_new = zeros(Ny, Nx);

    for i = 2:Nx-1
        for j = 2:Ny-1
            % Laplacian of pressure
            lap_P = (P(j,i+1) + P(j,i-1) - 2*P(j,i))/dx^2 + ...
                    (P(j+1,i) + P(j-1,i) - 2*P(j,i))/dy^2;

            % Wave equation: d²P/dt² = c²∇²P + source
            % Using finite difference in time
            if n == 1
                P_new(j,i) = P(j,i) + (c*dt)^2 * lap_P;
            else
                P_new(j,i) = 2*P(j,i) - P_prev(j,i) + (c*dt)^2 * lap_P;
            end

            % Add combustion source (modulated by instability)
            source = 1e4 * (1 + 0.3*A_inst) * exp(-((x(i)-x0)^2 + (y(j)-y0)^2)/0.005) * sin(omega0*t);
            P_new(j,i) = P_new(j,i) + dt^2 * source;
        end
    end

    % Boundary conditions
```

```
    % Walls: Neumann (dP/dn = 0)
    P_new(1,:) = P_new(2,:);       % Bottom
    P_new(Ny,:) = P_new(Ny-1,:);   % Top
    P_new(:,1) = P_new(:,2);       % Left (closed end)
    P_new(:,Nx) = 101325;          % Right (open to atmosphere)

    % Update pressure fields
    P_prev = P;
    P = P_new;

    % Store center pressure
    pressure_center(n) = P(round(Ny/2), round(Nx/2));

    % ========== Update plots ==========
    if mod(n, 5) == 0 || n == Nt
        % Update pressure surface
        set(h_p, 'ZData', P);
        set(h_p, 'CData', P);

        % Update instability surface
        set(h_phi, 'ZData', phi);
        set(h_phi, 'CData', phi);

        % Update time traces
        set(h_zplot, 'XData', time_points(1:n), 'YData', real(z_history(1:n)));
        set(h_pplot, 'XData', time_points(1:n), 'YData', pressure_center(1:n));

        % Update titles
        subplot(1,3,1);
        title(sprintf('Pressure at t = %.4f s', t));

        subplot(1,3,2);
        title(sprintf('Instability: |z| = %.3f', abs(z)));

        drawnow;
    end

    % Optional: Stop if unstable
    if abs(z) > 1.0
        fprintf('Large instability at t = %.4f s\n', t);
        break;
    end
end

fprintf('Simulation complete.\n');

%% ================== 6. POST-PROCESSING ============================
figure('Position', [100 100 900 400]);

% Plot 1: Bifurcation diagram
subplot(1,2,1);
mu_test = linspace(-0.2, 0.2, 50);
z_steady = zeros(size(mu_test));

for k = 1:length(mu_test)
    if mu_test(k) > 0
        z_steady(k) = sqrt(mu_test(k)/beta);
    else
        z_steady(k) = 0;
    end
end

plot(mu_test, z_steady, 'b-', 'LineWidth', 2);
hold on;
plot(mu_test, -z_steady, 'b--', 'LineWidth', 1);
xline(0, 'k--', 'LineWidth', 1);
xlabel('\mu (Control Parameter)');
ylabel('Steady State |z|');
title('Subcritical Hopf Bifurcation');
grid on;
legend('Stable', 'Unstable');

% Plot 2: Frequency spectrum
subplot(1,2,2);
Fs = 1/dt;
L = length(pressure_center);
NFFT = 2^nextpow2(L);
Y = fft(pressure_center - mean(pressure_center), NFFT)/L;
```

```
f = Fs/2*linspace(0,1,NFFT/2+1);

plot(f, 2*abs(Y(1:NFFT/2+1)), 'r-', 'LineWidth', 1.5);
xlabel('Frequency (Hz)');
ylabel('|P(f)|');
title('Pressure Spectrum');
grid on;
xlim([0, 5000]);

%% ==================== 7. PARAMETER ANALYSIS ==========================
fprintf('\n=== Simulation Results ===\n');
fprintf('Final instability amplitude: |z| = %.4f\n', abs(z));
fprintf('Pressure oscillation amplitude: %.1f kPa\n', ...
    (max(pressure_center) - min(pressure_center))/1000);
fprintf('Dominant frequency: %.1f Hz\n', f(find(2*abs(Y(1:NFFT/2+1)) == max(2*abs(Y(1:NFFT/2+1))), 1)));

% Show geometry effects from paper
fprintf('\n=== From Paper (Optimal L/D ratios) ===\n');
fprintf('Exhaust pipe: 29\n');
fprintf('Combustion chamber: 1.25\n');
fprintf('Intake pipe: 3.5\n');
```